

GoalKeeper

Low-Level Design Document

Frontend Architecture + Backend Architecture | v1.0

Document Version	1.0 — Based on GoalKeeper PRD v1.0
Status	Engineering Draft
Date	February 2025
Authors	Engineering Team
Audience	Frontend Engineers, Backend Engineers, DevOps, Tech Leads
Prerequisite	GoalKeeper PRD v1.0 must be read before this document

Purpose

This Low-Level Design document provides the complete implementation blueprint for GoalKeeper. It covers: React PWA component tree and state architecture, routing and navigation design, service layer patterns, all Go backend service internal designs, data access patterns, inter-service communication, background job design, caching strategy, security implementation, and observability setup. Every decision is traceable to a requirement in the PRD.

1. System Overview & Design Principles

1.1 Architecture Summary

GoalKeeper is built as a modular monorepo containing a React PWA frontend and a set of loosely coupled Go microservices. The services communicate synchronously via REST/gRPC for request-response operations and asynchronously via NATS JetStream for event-driven flows such as XP distribution, notifications, and leaderboard updates.

Core Design Principles

Mobile-First UI: Every component is designed at 375px, then scaled up. Desktop is enhancement, not the baseline.

Offline-First PWA: IndexedDB queues mutations when offline. Service Worker intercepts fetch and syncs on reconnect.

Stateless Backend: All Go service pods carry zero in-memory session state. JWT + Redis for auth state.

Event-Driven Async: XP awards, notifications, leaderboard updates, buddy rotations — all async via NATS JetStream.

Row-Level Security: Every DB query is scoped to the authenticated user via Postgres RLS policies.

Schema-First API: OpenAPI 3.1 spec is the single source of truth. Client and server types are generated from it.

Observability-Native: Every service emits structured logs (JSON), metrics (Prometheus), and traces (OTLP/Jaeger).

1.2 Repository Structure

```
goalkeeper/
├── apps/
│   ├── web/           # React PWA (Vite)
│   └── admin/         # Internal admin dashboard (React)
├── services/
│   ├── users/         # Go: user profiles, auth integration
│   ├── goals/         # Go: goals, library, milestones
│   ├── tasks/         # Go: tasks, recurrence engine, state machine
│   ├── community/     # Go: communities, membership, buddy system
│   ├── gamification/  # Go: XP, coins, levels, streaks
│   ├── verification/  # Go: proof submission, review workflow
│   ├── store/         # Go: reward store, inventory, purchases
│   ├── notification/  # Go: push, email, in-app feed
│   ├── search/        # Go: Typesense indexing, query routing
│   └── analytics/     # Go: event ingestion, aggregation, AI feed
├── packages/
│   └── api-types/     # Shared TypeScript types (generated from
OpenAPI)
├── ui/               # Shared React component library
├── proto/            # Protobuf definitions for internal gRPC
├── infra/
│   ├── k8s/          # Kubernetes manifests (HPA, Services, Ingress)
│   └── terraform/    # Cloud infra (GCP/AWS)
```

```
| └─ migrations/           # Flyway SQL migration files
└─ tools/
    └─ openapi-gen/        # Schema → TypeScript + Go type generation
└─ .github/workflows/      # CI/CD pipelines
```

2. Frontend Low-Level Design

2.1 Technology Stack Detail

Package	Version	Purpose
react	19.x	UI rendering, Suspense, concurrent features
vite	6.x	Build tool, HMR, PWA plugin (vite-plugin-pwa)
typescript	5.x	Static typing across entire frontend
react-router-dom	7.x	Client-side routing, nested layouts, loaders
zustand	5.x	Lightweight global state (auth, user, UI flags)
@tanstack/react-query	5.x	Server state — fetch, cache, sync, mutation
tailwindcss	4.x	Utility-first CSS, dark mode, responsive variants
@radix-ui/react-*	latest	Accessible headless primitives (Dialog, Sheet, etc.)
dnd-kit	6.x	Drag-and-drop for calendar task cards
react-big-calendar	1.x	Calendar grid base (week/day/month views)
date-fns	3.x	Date arithmetic, formatting, timezone handling
react-hook-form + zod	7.x / 3.x	Form state + schema-based validation
axios	1.x	HTTP client with interceptors for JWT refresh
workbox	7.x	Service Worker strategies, offline queue, sync
idb	8.x	IndexedDB wrapper for offline mutation queue
firebase/app + messaging	10.x	FCM push notifications
recharts	2.x	Progress charts, analytics dashboards
framer-motion	11.x	Micro-animations, page transitions, celebrations
lucide-react	latest	Icon set

2.2 Application Shell & Routing

Route Tree

```
/ (AppShell - persistent layout: bottom nav, top bar)
├── /                               → <Dashboard />
├── /goals
│   ├── /                           → <GoalList />
│   ├── /new                         → <GoalCreate />
│   ├── /:goalId                     → <GoalDetail />
│   └── /:goalId/edit                 → <GoalEdit />
├── /tasks
│   ├── /new?goalId=                 → <TaskCreate />
│   └── /:taskId/edit                → <TaskEdit />
```

```
|— /calendar          → <CalendarView />
|— /community
|   |— /              → <CommunityHub />
|   |— /new           → <CommunityCreate />
|   |— /:communityId
|       |— /          → <CommunityDetail />
|       |— /leaderboard → <Leaderboard />
|       |— /members    → <MemberList />
|       |— /admin      → <CommunityAdmin /> (role-guarded)
|— /buddy             → <BuddyPanel />
|— /proofs
|   |— /pending       → <ProofQueue />
|   |— /:proofId      → <ProofReview />
|— /store             → <RewardStore />
|— /library           → <GoalLibrary />
|— /notifications    → <NotificationCenter />
|— /analytics         → <AnalyticsDashboard />
|— /profile/:userId   → <UserProfile />
|— /settings          → <Settings />
|— /auth
|   |— /login         → <Login />
|   |— /register      → <Register />
|   |— /callback      → <AuthCallback /> (OAuth redirect)
```

Route Guards

```
// apps/web/src/router/guards.tsx
export const AuthGuard = ({ children }) => {
  const { user, isLoading } = useAuthStore();
  if (isLoading) return <SplashScreen />;
  if (!user) return <Navigate to="/auth/login" replace />;
  return children;
};

export const RoleGuard = ({ role, children }) => {
  const { communityRole } = useCommunityStore();
  if (communityRole !== role) return <Navigate to=".." replace />;
  return children;
};
```

2.3 State Architecture

Store Design (Zustand)

GoalKeeper uses three Zustand stores for global client-owned state. Server state (goals, tasks, community data) lives entirely in React Query cache. Zustand is reserved for state that has no server representation.

```
// stores/auth.store.ts — Authentication state
interface AuthStore {
```

```
user:      User | null;
accessToken: string | null;
isLoading:  boolean;
login:      (creds: LoginPayload) => Promise<void>;
logout:     () => void;
refreshToken: () => Promise<void>;
}

// stores/ui.store.ts - Transient UI state
interface UIStore {
  theme:      'light' | 'dark' | 'system';
  bottomSheetOpen: boolean;
  activeModal: ModalKey | null;
  calendarView: 'week' | 'day' | 'month';
  calendarDate: Date;
  taskColorMode: 'goal' | 'priority' | 'status';
  notificationCount: number;
  setTheme:     (t: Theme) => void;
  openModal:    (key: ModalKey) => void;
  closeModal:   () => void;
}

// stores/gamification.store.ts - Optimistic XP/coins (animated locally)
interface GamificationStore {
  xp:      number;
  coins:   number;
  level:   number;
  streak:  number;
  pendingXP: number; // XP earned this session, pre-animation
  addXP:    (amount: number) => void;
  addCoins: (amount: number) => void;
  syncFromServer: (snapshot: GamificationSnapshot) => void;
}
```

React Query Configuration

```
// lib/queryClient.ts
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000, // 1 min - data considered fresh
      gcTime: 300_000,   // 5 min - cache kept in memory
      retry: 2,
      refetchOnWindowFocus: true,
    },
    mutations: {
      onError: (err) => toast.error(formatApiError(err)),
    },
  },
});

// Key factory - centralised, type-safe query keys
export const queryKeys = {
```

```
goals:      { all: ['goals'], byId: (id) => ['goals', id] },
tasks:      { all: ['tasks'], byGoal: (g) => ['tasks','goal',g],
              calendar: (f,t)=> ['tasks','calendar',f,t] },
community:  { all: ['communities'], byId: (id) => ['communities',id],
              leaderboard:(id,window)=>['communities',id,'lb',window] },
proofs:     { pending: ['proofs','pending'], byId:(id)=>['proofs',id] },
gamification:{ me: ['gamification','me'] },
store:      { items:(cat,rar)=>['store','items',cat,rar] },
notifications:{ feed: ['notifications','feed'] },
};
```

2.4 Component Architecture

Component Taxonomy

Layer	Location	Naming Convention	Examples
Design System	packages/ui/	<GK*>	GKButton, GKCard, GKXPBar, GKBadge, GKAvatar
Feature Components	apps/web/src/features/*/	<Feature*>	GoalCard, TaskCard, BuddyPanel, ProofUpload
Screen Components	apps/web/src/screens/*/	<*Screen> or <*View>	DashboardScreen, CalendarView, GoalDetailScreen
Layout Components	apps/web/src/layouts/	<*Layout>	AppShell, AuthLayout, FullPageLayout
Form Components	apps/web/src/forms/*/	<*Form>	GoalForm, TaskForm, CommunityForm
Hooks	apps/web/src/hooks/	use*	useGoals, useTasks, useCalendar, useProofUpload

2.4.1 Dashboard Screen Component Tree

```
<DashboardScreen>
  <DashboardTopBar>                                // XP bar, coins, notification bell
    <GKXPBar level={lvl} xp={xp} />
    <GKCoinCounter coins={coins} />
    <NotificationBell count={n} />
  </DashboardTopBar>

  <TodayTasksStrip>                                // Horizontal scroll, sorted by time
    <TaskCard task={t} variant='compact' onComplete={...} />
  </TodayTasksStrip>

  <ActiveGoalsGrid>                                // 2-col grid
    <GoalCard goal={g} variant='dashboard' />
```

```
</ActiveGoalsGrid>

<BuddyPanelWidget>                                // Collapsible card
  <GKAvatar user={buddy} />
  <BuddyProgressBar completion={pct} />
  <VerificationDueTimer expiresAt={t} />
</BuddyPanelWidget>

<StreakWidget streak={n} record={r} />

<CommunityQuickLinks communities={top3} />
</DashboardScreen>
```

2.4.2 Calendar View Component Tree

```
<CalendarView>
  <CalendarHeader>
    <ViewToggle active={view} onChange={setView} />    // week|day|month
    <DateNavigator date={date} onNav={setDate} />
    <TagFilterChips tags={activeTags} />
    <ColorModeToggle mode={colorMode} />
  </CalendarHeader>

  <GoalTimelineBar goals={activeGoals} dateRange={range} />

  <DndContext onDragEnd={handleReschedule} collisionDetection={closestCenter}>
    <BigCalendar
      view={view}
      date={date}
      events={taskInstances}
      components={{
        event: DraggableTaskCard,
        dateCellWrapper: DroppableTimeSlot,
      }}
    />
  </DndContext>

  <RescheduleModal                                // Opens on drop for recurring tasks
    open={rescheduleOpen}
    onChoice={applyReschedule}
  />
</CalendarView>
```

2.4.3 Task Configuration Form

```
<TaskForm goalId={goalId} taskId={taskId?}>
  <Controller name='title' />                      // TextInput
  <Controller name='description' />                 // RichTextArea
  <Controller name='goalId' />                     // SearchableSelect
  <TypeToggle name='type' />                       // 'one_time' | 'recurring'
```



```
{type==='recurring' && (  
  <RecurrenceBuilder>  
    <FrequencySelect /> // daily|weekly|monthly|custom  
    {freq==='weekly' && <DayPicker />}  
    {freq==='monthly' && <DatePicker />}  
    {advanced && <CronInput />}  
  </RecurrenceBuilder>  
)}  
  
<TimeScheduler name='scheduledAt' /> // TimePicker + Duration  
<ReminderOffset name='reminderMinutes' />  
<TagInput name='tags' communityId={cid} />  
<ProofConfig name='proof' />  
<XPField name='xpValue' locked={adminLocked} />  
<PriorityToggle name='priority' />  
<MandatoryToggle name='isMandatory' />  
<RecurrencePreviewCalendar rule={watchedRule} />  
</TaskForm>
```

2.5 Custom Hooks Design

useGoals Hook

```
// hooks/useGoals.ts  
export const useGoals = (filters?: GoalFilters) => {  
  const query = useQuery({  
    queryKey: queryKeys.goals.all,  
    queryFn: () => api.goals.list(filters),  
  });  
  const createMutation = useMutation({  
    mutationFn: api.goals.create,  
    onSuccess: (newGoal) => {  
      queryClient.setQueryData(queryKeys.goals.all, (old) =>  
        old ? [newGoal, ...old] : [newGoal]  
      );  
      queryClient.invalidateQueries({ queryKey: queryKeys.goals.all });  
    },  
  });  
  return { ...query, createGoal: createMutation.mutate };  
};
```

useCalendar Hook

```
// hooks/useCalendar.ts  
export const useCalendar = (dateRange: DateRange) => {  
  const { data: taskInstances } = useQuery({  
    queryKey: queryKeys.tasks.calendar(dateRange.from, dateRange.to),  
    queryFn: () => api.tasks.getCalendar(dateRange),  
    staleTime: 30_000,  
  });  
};
```

```
});

const rescheduleMutation = useMutation({
  mutationFn: ({ taskId, newDate, scope }) =>
    api.tasks.reschedule(taskId, { newDate, scope }),
  onMutate: async (vars) => {
    // Optimistic update - move card immediately
    await queryClient.cancelQueries({ queryKey:
queryKeys.tasks.calendar(...) });
    const snapshot = queryClient.getQueryData(...);
    queryClient.setQueryData(..., optimisticReschedule(snapshot, vars));
    return { snapshot };
  },
  onError: (_err, _vars, ctx) => {
    queryClient.setQueryData(..., ctx.snapshot); // rollback
  },
});

return { taskInstances, reschedule: rescheduleMutation.mutate };
};
```

useOfflineQueue Hook

```
// hooks/useOfflineQueue.ts
// Queues mutations to IndexedDB when offline; replays on reconnect
export const useOfflineQueue = () => {
  const db = useIDB('goalkeeper-offline', OFFLINE_STORE_V1);

  const enqueue = async (op: OfflineOperation) => {
    await db.put('mutations', { ...op, timestamp: Date.now() });
  };

  const flush = async () => {
    const ops = await db.getAll('mutations');
    for (const op of ops.sort((a,b) => a.timestamp - b.timestamp)) {
      try {
        await api[op.service][op.method](op.payload);
        await db.delete('mutations', op.id);
      } catch (e) {
        if (!isRetryable(e)) await db.delete('mutations', op.id);
      }
    }
  };

  useEffect(() => {
    window.addEventListener('online', flush);
    return () => window.removeEventListener('online', flush);
  }, []);

  return { enqueue };
};
```

2.6 API Client Layer

```
// lib/api/client.ts - Axios instance with JWT interceptor
const apiClient = axios.create({ baseURL: import.meta.env.VITE_API_URL });

apiClient.interceptors.request.use(config => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

apiClient.interceptors.response.use(
  res => res,
  async err => {
    if (err.response?.status === 401 && !err.config._retry) {
      err.config._retry = true;
      await useAuthStore.getState().refreshToken();
      return apiClient(err.config);
    }
    return Promise.reject(err);
  }
);

// lib/api/goals.ts - Resource-specific module
export const goalsApi = {
  list: (f?) => apiClient.get('/v1/goals', { params: f
}).then(r=>r.data),
  getById: (id) => apiClient.get(`/v1/goals/${id}`).then(r=>r.data),
  create: (payload) => apiClient.post('/v1/goals', payload).then(r=>r.data),
  update: (id,p) => apiClient.patch(`/v1/goals/${id}`, p).then(r=>r.data),
  delete: (id) => apiClient.delete(`/v1/goals/${id}`).then(r=>r.data),
};
```

2.7 PWA & Service Worker Design

Workbox Strategy Map

Resource Type	Caching Strategy	Details
App Shell (HTML/JS/CSS)	Cache-First	Served from cache. Updated in background via SW update lifecycle.
API GET requests	Network-First (5s timeout)	Falls back to stale cache. Stale response shown with 'offline' badge.
Static Assets (icons, fonts)	Cache-First with expiry	Cached for 30 days. Versioned via content hash.
Media (proof photos, avatars)	Stale-While-Revalidate	Serve cached, fetch update in background.
Push Notifications	Background Sync	Queued via IndexedDB, replayed when online.

Task completion mutation	Background Sync queue	IDB mutation record; SW replays on reconnect.
--------------------------	-----------------------	---

Offline Mutation Flow

```
// sw.ts - Service Worker (Workbox + custom sync logic)
self.addEventListener('sync', (event) => {
  if (event.tag === 'goalkeeper-mutation-sync') {
    event.waitUntil(replayOfflineQueue());
  }
});

async function replayOfflineQueue() {
  const db = await openDB('goalkeeper-offline', 1);
  const mutations = await db.getAll('mutations');
  for (const m of mutations) {
    const resp = await fetch(m.url, {
      method: m.method,
      headers: { 'Content-Type': 'application/json', 'Authorization': m.token },
      body: JSON.stringify(m.body),
    });
    if (resp.ok) await db.delete('mutations', m.id);
  }
}
```

2.8 Gamification UI Patterns

XP Award Animation Flow

```
// components/celebrations/XPReward.tsx
// Triggered by gamification.store.pendingXP > 0
export const XPRewardOverlay = () => {
  const { pendingXP, clearPending } = useGamificationStore();
  return (
    <AnimatePresence>
      {pendingXP > 0 && (
        <motion.div
          initial={{ y: 60, opacity: 0 }}
          animate={{ y: 0, opacity: 1 }}
          exit={{ y: -60, opacity: 0 }}
          onAnimationComplete={clearPending}
        >
          <span>+{pendingXP} XP</span>
          <GKXPBar animated />
          {levelUp && <LevelUpCelebration />}
        </motion.div>
      )}
    </AnimatePresence>
  );
};
```

Drag-and-Drop Calendar Reschedule Flow

```
// 1. User grabs task card → DndContext fires onDragStart
// 2. DroppableTimeSlot highlights on DragOver (CSS class 'drop-target')
// 3. User drops → DndContext fires onDragEnd
const handleReschedule = (event: DragEndEvent) => {
  const { active, over } = event;
  if (!over) return;
  const task = getTaskById(active.id);
  const newSlotDate = parseSlotId(over.id);          // 'slot-2025-03-01T09:00'
  if (task.type === 'recurring') {
    setRescheduleCtx({ task, newDate: newSlotDate });
    openModal('reschedule-choice');                  // modal: this/future/all
  } else {
    reschedule({ taskId: task.id, newDate: newSlotDate, scope: 'one' });
  }
};
```

3. Backend Low-Level Design — Overview

3.1 Go Service Conventions

Every Go service follows the same internal structure. This ensures consistency, ease of onboarding, and predictable tooling.

```
service/
├── cmd/server/main.go           // Entrypoint: wire deps, start HTTP + gRPC
servers
├── internal/
│   └── handler/                // HTTP handlers (echo/gin) - thin, no
business logic
├── service/                    // Business logic layer - pure, testable
│   ├── repository/            // Data access - only SQL/Redis/external calls
│   ├── domain/                // Domain types, errors, constants
│   ├── middleware/            // Auth, logging, metrics, recovery
│   └── events/                 // NATS publisher/subscriber interfaces
├── pkg/                        // Exported utility packages (reusable)
├── migrations/                // Service-scoped SQL migrations
└── Dockerfile
```

Layered Call Flow

```
HTTP Request
→ Middleware (auth JWT → attach userID to ctx)
→ Handler (parse + validate input, call service)
→ Service (business logic, domain rules, call repository)
→ Repository (SQL query via pgx, Redis via go-redis)
→ Response (handler serialises to JSON, sets status code)

// Async side effects
→ Service publishes event to NATS JetStream
→ notification-service consumes event, dispatches push/email
→ gamification-service consumes event, awards XP/coins
```

3.2 Shared Infrastructure Packages

Package	Purpose	Implementation
pkg/auth	JWT validation, claims extraction	RS256, public key fetched from auth provider JWKS endpoint
pkg/db	PostgreSQL pool + helpers	pgxpool, transaction helper, RLS context injection
pkg/cache	Redis client + helpers	go-redis v9, typed wrappers for ZSET, hash, string ops
pkg/nats	NATS JetStream pub/sub	Typed message envelopes, retry policy, dead-letter subject

pkg/logger	Structured JSON logging	zerolog, request-scoped fields (trace_id, user_id, service)
pkg/tracer	OpenTelemetry tracing	OTLP exporter to Jaeger, automatic span propagation via ctx
pkg/validator	Request validation	go-validator v10, custom rule registration
pkg/errcode	Domain error registry	Typed errors mapping to HTTP status + RFC 7807 body
pkg/pagination	Cursor pagination	Opaque cursor = base64(JSON{id,created_at}), consistent across all list endpoints

4. users-service

4.1 Responsibilities

- Sync user identity from auth provider (Supabase/Clerk) webhook on first login.
- Own the users table and expose profile CRUD.
- Manage cosmetic inventory (equipped avatar, title, background).
- Expose /users/me endpoint as the central user data aggregation point.

4.2 Handler → Service → Repository

```
// handler/user_handler.go
func (h *UserHandler) GetMe(c echo.Context) error {
    userID := middleware.UserIDFromCtx(c.Request().Context())
    user, err := h.svc.GetUser(c.Request().Context(), userID)
    if err != nil { return errcode.ToHTTP(err) }
    return c.JSON(200, toUserResponse(user))
}

// service/user_service.go
func (s *UserService) GetUser(ctx context.Context, id uuid.UUID)
(*domain.User, error) {
    user, err := s.repo.FindByID(ctx, id)
    if errors.Is(err, pgx.ErrNoRows) { return nil, errcode.ErrNotFound }
    return user, err
}

// repository/user_repo.go
func (r *UserRepo) FindByID(ctx context.Context, id uuid.UUID) (*domain.User,
error) {
    row := r.pool.QueryRow(ctx,
        `SELECT id, email, display_name, avatar_url, level, total_xp,
        coins, streak_current, streak_best, solo_mode
        FROM users WHERE id = $1`, id)
    var u domain.User
    if err := row.Scan(&u.ID, &u.Email, ...); err != nil { return nil, err }
    return &u, nil
}
```

4.3 Auth Provider Webhook Sync

```
// POST /internal/webhooks/auth (internal endpoint, not public)
func (h *UserHandler) HandleAuthWebhook(c echo.Context) error {
    var event AuthWebhookEvent
    if err := c.Bind(&event); err != nil { return c.JSON(400, err) }
    if event.Type == 'user.created' {
        return h.svc.UpsertFromProvider(c.Request().Context(), event.Data)
    }
}
```



```
    return c.NoContent(200)
}

func (s *UserService) UpsertFromProvider(ctx context.Context, d
ProviderUserData) error {
    _, err := s.repo.Upsert(ctx, &domain.User{
        ID:          d.ID,          // Provider user ID == our UUID
        Email:       d.Email,
        DisplayName: d.Name,
        AvatarURL:   d.AvatarURL,
    })
    return err
}
```

5. goals-service

5.1 Responsibilities

- CRUD for user goals and community goals.
- Progress calculation engine: completed task instances / total scheduled task instances within goal timeline.
- Goal Library: indexing, adoption (fork), search routing.
- Milestone management within a goal timeline.

5.2 Progress Calculation

```
// service/progress.go
// Called on every task state change via NATS event
func (s *GoalService) RecalculateProgress(ctx context.Context, goalID
uuid.UUID) error {
    // Count all task instances that fall within goal.start_date →
goal.target_date
    total, completed, err := s.repo.GetTaskInstanceCounts(ctx, goalID)
    if err != nil { return err }
    pct := 0.0
    if total > 0 { pct = float64(completed) / float64(total) * 100 }
    return s.repo.UpdateProgress(ctx, goalID, pct)
}

// SQL used by GetTaskInstanceCounts
-- Counts from task_instances (expanded recurring tasks) not tasks table
SELECT
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE status IN
('completed_verified','completed_unverified')) AS completed
FROM task_instances
WHERE goal_id = $1
    AND scheduled_at BETWEEN (SELECT start_date FROM goals WHERE id=$1)
    AND (SELECT target_date FROM goals WHERE id=$1);
```

5.3 Goal Library Fork

```
// POST /v1/goals/library/:id/adopt
func (s *GoalService) AdoptLibraryGoal(ctx context.Context, userID, libGoalID
uuid.UUID) (*domain.Goal, error) {
    lib, err := s.repo.GetLibraryGoal(ctx, libGoalID)
    if err != nil { return nil, err }
    // Deep copy: new goal + new tasks with new IDs, user ownership
    newGoal := lib.Fork(userID)
    if err := s.repo.CreateGoalWithTasks(ctx, newGoal); err != nil { return nil,
err }
    // Increment adoption count async
```

```
s.events.Publish(ctx, 'library.goal.adopted', LibraryAdoptedEvent{LibGoalID:
libGoalID})
return newGoal, nil
}
```

6. tasks-service

6.1 Recurrence Engine

Recurring tasks are stored as a single tasks row with a `recurrence_rule` JSONB field. A background job (recurrence expander) materialises concrete `task_instances` rows ahead of time (rolling 90-day window). The UI always works with `task_instances`, never raw tasks.

```
// domain/recurrence.go - RecurrenceRule structure
type RecurrenceRule struct {
    Frequency string `json:'freq'` // daily|weekly|monthly|custom
    Interval  int   `json:'interval'` // every N units (default 1)
    ByDay     []string `json:'byday'` // ['MO','WE','FR'] for weekly
    ByMonthDay []int  `json:'bymonthday'` // [1,15] for monthly
    Until     *time.Time `json:'until'` // end date, nil = follow goal
    target_date
    Cron      string `json:'cron'` // raw cron expression for 'custom'
}

// service/expander.go - Expansion Job (runs every 24h per service)
func (s *TaskService) ExpandRecurringTasks(ctx context.Context) error {
    tasks, err := s.repo.GetRecurringTasksDueExpansion(ctx, 90) // expand 90
    days ahead
    for _, t := range tasks {
        instances := t.RecurrenceRule.Expand(t.LastExpandedAt,
            time.Now().AddDate(0,0,90))
        if err := s.repo.BulkInsertInstances(ctx, instances); err != nil {
            log.Error(err)
        }
        s.repo.UpdateLastExpanded(ctx, t.ID, time.Now())
    }
    return nil
}
```

6.2 Task State Machine

From State	Event	To State	Side Effects
scheduled	window opens (cron)	pending	Reminder push notification dispatched
pending	user marks complete (no proof)	completed_verified	XP event published to gamification-svc
pending	user marks complete (proof req.)	completed_unverified	Proof record created; buddy notified
completed_unverified	buddy approves	completed_verified	XP event published; buddy XP event published
completed_unverified	buddy rejects	failed	No XP; appeal option opened
completed_unverified	verification window expires	completed_verified (auto)	Auto-approve; buddy verification_score -5

pending	user skips	skipped	No XP; streak check; buddy notified
pending	window expires (cron)	missed	Streak broken; buddy notified
failed	user appeals, mod approves	completed_verified	XP retroactively awarded

6.3 Task Completion Handler

```
// POST /v1/tasks/:id/complete
func (s *TaskService) CompleteTask(ctx context.Context, req
CompleteTaskRequest) error {
    instance, err := s.repo.GetInstance(ctx, req.TaskInstanceID)
    if err != nil || instance.Status != 'pending' {
        return errcode.ErrInvalidTransition
    }
    // Determine next status
    nextStatus := 'completed_verified'
    if instance.ProofRequired {
        nextStatus = 'completed_unverified'
    }
    if err := s.repo.SetStatus(ctx, instance.ID, nextStatus); err != nil {
return err }
    // Publish events
    if nextStatus == 'completed_verified' {
        s.events.Publish(ctx, 'task.completed', TaskCompletedEvent{
            UserID: instance.UserID, TaskID: instance.ID,
            XPValue: instance.XPValue, CoinValue: instance.CoinValue,
        })
    } else {
        s.events.Publish(ctx, 'proof.required', ProofRequiredEvent{
            TaskInstanceID: instance.ID, BuddyID: instance.BuddyID,
            ExpiresAt: time.Now().Add(time.Duration(instance.VerificationWindowHrs)
* time.Hour),
        })
    }
    s.events.Publish(ctx, 'goal.progress.recalc', GoalProgressEvent{GoalID:
instance.GoalID})
    return nil
}
```

7. community-service

7.1 Buddy Assignment Algorithm

```
// service/buddy_assigner.go
func (s *CommunityService) AssignBuddy(ctx context.Context, communityID,
goalID, userID uuid.UUID) error {
    // 1. Fetch all active participants in this community+goal without a buddy
    pool, err := s.repo.GetUnpairedParticipants(ctx, communityID, goalID,
userID)
    if err != nil { return err }
    if len(pool) == 0 {
        return s.repo.AddToWaitingQueue(ctx, communityID, goalID, userID)
    }
    // 2. Score candidates (round-robin by default; smart if feature flag on)
    buddy := s.scoreCandidates(userID, pool, s.cfg.SmartMatchEnabled)
    // 3. Create buddy_assignment record
    rotationEnd := time.Now().Add(time.Duration(s.cfg.BuddyRotationDays) * 24 *
time.Hour)
    if err := s.repo.CreateBuddyAssignment(ctx, &domain.BuddyAssignment{
        CommunityID: communityID,
        GoalID: goalID,
        UserID: userID,
        BuddyID: buddy.ID,
        RotationStart: time.Now(),
        RotationEnd: rotationEnd,
    }); err != nil { return err }
    // 4. Notify both parties
    s.events.Publish(ctx, 'buddy.assigned', BuddyAssignedEvent{UserID: userID,
BuddyID: buddy.ID})
    return nil
}

func (s *CommunityService) scoreCandidates(requester uuid.UUID, pool
[]domain.User, smart bool) domain.User {
    if !smart { return pool[0] } // round-robin: first unassigned
    // Smart: score by |level diff| + |timezone offset|, pick min score
    requesterProfile := s.repo.GetProfile(requester)
    scored := lo.Map(pool, func(u domain.User, _ int) ScoredUser {
        levelDiff := abs(u.Level - requesterProfile.Level)
        tzDiff := abs(u.TimezoneOffset - requesterProfile.TimezoneOffset)
        return ScoredUser{User: u, Score: levelDiff*2 + tzDiff}
    })
    return lo.MinBy(scored, func(a, b ScoredUser) bool { return a.Score <
b.Score }).User
}
```

7.2 Buddy Rotation Scheduler

```
// Background job: runs every hour, checks for expired rotations
```

```
func (s *CommunityService) ProcessExpiredRotations(ctx context.Context) error {
    expired, err := s.repo.GetExpiredBuddyAssignments(ctx)
    if err != nil { return err }
    for _, assignment := range expired {
        // Mark old assignment inactive
        s.repo.DeactivateAssignment(ctx, assignment.ID)
        // Reassign both users to new buddies from pool
        s.AssignBuddy(ctx, assignment.CommunityID, assignment.GoalID,
assignment.UserID)
        s.AssignBuddy(ctx, assignment.CommunityID, assignment.GoalID,
assignment.BuddyID)
        // Notify both
        s.events.Publish(ctx, 'buddy.rotated', BuddyRotatedEvent{
            OldAssignment: assignment,
        })
    }
    return nil
}
```

7.3 Leaderboard Design

```
// Redis ZSET key pattern: lb:{communityId}:{goalId}:{window}
// window: 'all' | '2025-03' (monthly) | '2025-W10' (weekly)

// gamification-service publishes 'xp.awarded' event
// community-service subscribes and updates ZSET:
func (s *CommunityService) HandleXPAwarded(ctx context.Context, e
XPAwardedEvent) error {
    if e.CommunityID == uuid.Nil { return nil } // solo, skip
    pipe := s.redis.Pipeline()
    for _, window := range []string{'all', monthWindow(), weekWindow()} {
        key := fmt.Sprintf('lb:%s:%s:%s', e.CommunityID, e.GoalID, window)
        pipe.ZIncrBy(ctx, key, float64(e.Amount), e.UserID.String())
        pipe.Expire(ctx, key, 90*24*time.Hour)
    }
    _, err := pipe.Exec(ctx)
    return err
}

// Leaderboard query: GET
/v1/communities/:id/leaderboard?window=month&limit=50
func (s *CommunityService) GetLeaderboard(ctx context.Context, communityID
uuid.UUID, window string, limit int) ([]LeaderboardEntry, error) {
    key := fmt.Sprintf('lb:%s:all:%s', communityID, window) // simplified
    members, err := s.redis.ZRevRangeWithScores(ctx, key, 0, int64(limit-
1)).Result()
    if err != nil { return nil, err }
    // Hydrate user details from users-service (gRPC)
    return s.hydrateLeaderboard(ctx, members), nil
}
```

8. gamification-service

8.1 XP Engine Design

The gamification service is a pure event consumer. It subscribes to NATS subjects and applies XP/coin award logic. It never gets called directly from user-facing endpoints — all awards are triggered by domain events from other services.

```
// events/consumers.go - Subject subscriptions
var subscriptions = []EventSub{
    { Subject: 'task.completed',    Handler: handleTaskCompleted },
    { Subject: 'proof.approved',    Handler: handleProofApproved },
    { Subject: 'goal.completed',    Handler: handleGoalCompleted },
    { Subject: 'streak.milestone',  Handler: handleStreakMilestone },
}

// service/xp_engine.go
func handleTaskCompleted(ctx context.Context, e TaskCompletedEvent) error {
    // 1. Fetch current streak for multiplier
    streak, _ := xpSvc.repo.GetStreak(ctx, e.UserID)
    multiplier := streakMultiplier(streak)
    // 2. Apply solo penalty if applicable
    if e.IsSolo { multiplier *= xpSvc.cfg.SoloPenaltyRatio } // default 0.60
    // 3. Compute final XP
    finalXP := int(math.Round(float64(e.XPValue) * multiplier))
    finalCoins := int(math.Round(float64(e.CoinValue) * multiplier))
    // 4. Write to xp_ledger + update users table
    xpSvc.repo.AwardXP(ctx, e.UserID, finalXP, 'task_completion', e.TaskID,
e.CommunityID)
    xpSvc.repo.IncrCoins(ctx, e.UserID, finalCoins)
    // 5. Check level up
    xpSvc.checkAndApplyLevelUp(ctx, e.UserID)
    // 6. Update streak
    xpSvc.repo.UpdateStreak(ctx, e.UserID, time.Now())
    // 7. Publish result for UI toast
    xpSvc.events.Publish(ctx, 'xp.awarded', XPAwardedEvent{
        UserID: e.UserID, Amount: finalXP, Coins: finalCoins,
        Multiplier: multiplier, CommunityID: e.CommunityID,
    })
    return nil
}

func streakMultiplier(streak int) float64 {
    switch {
    case streak >= 30: return 2.0
    case streak >= 14: return 1.5
    case streak >= 7:  return 1.25
    case streak >= 3:  return 1.1
    default:           return 1.0
    }
}
```


8.2 Level Threshold Computation

```
// service/levels.go
// Level = 1 + floor((-1 + sqrt(1 + 8*XP/BASE)) / 2)
// Produces: L1:0, L2:500, L3:1500, L4:3000, L5:5000, L10:22500...
const BASE_XP = 250.0

func XPToLevel(totalXP int) int {
    if totalXP <= 0 { return 1 }
    level := int((-1 + math.Sqrt(1+8*float64(totalXP)/BASE_XP)) / 2)
    return max(1, level)
}

func (s *GamificationService) checkAndApplyLevelUp(ctx context.Context, userID
uuid.UUID) {
    user, _ := s.repo.GetUserSnapshot(ctx, userID)
    newLevel := XPToLevel(user.TotalXP)
    if newLevel > user.Level {
        bonusCoins := newLevel * 100    // 100 coins per level
        s.repo.SetLevel(ctx, userID, newLevel)
        s.repo.IncrCoins(ctx, userID, bonusCoins)
        s.events.Publish(ctx, 'user.levelup', LevelUpEvent{
            UserID: userID, OldLevel: user.Level, NewLevel: newLevel, BonusCoins:
bonusCoins,
        })
    }
}
```

9. verification-service

9.1 Proof Submission Flow

```
// POST /v1/tasks/:id/complete (multipart/form-data when proof required)
func (s *VerificationService) SubmitProof(ctx context.Context, req
ProofSubmitRequest) (*domain.ProofSubmission, error) {
    // 1. Validate task is in pending state and belongs to requesting user
    instance, err := s.tasksClient.GetInstance(ctx, req.TaskInstanceID)
    if err != nil || instance.UserID != req.UserID { return nil,
errcode.ErrForbidden }
    // 2. Upload media to object storage (S3 pre-signed PUT)
    var mediaURLs []string
    for _, file := range req.Files {
        url, err := s.storage.Upload(ctx, StorageUploadRequest{
            Bucket: 'proofs',
            Key:    fmt.Sprintf('%s/%s/%s', req.UserID, req.TaskInstanceID,
file.Name),
            Body:    file.Reader,
            ContentType: file.ContentType,
        })
        if err != nil { return nil, err }
        mediaURLs = append(mediaURLs, url)
    }
    // 3. Create proof_submissions record
    expiresAt := time.Now().Add(time.Duration(instance.VerificationWindowHrs) *
time.Hour)
    proof, err := s.repo.CreateProof(ctx, &domain.ProofSubmission{
        TaskInstanceID: req.TaskInstanceID,
        SubmitterID:     req.UserID,
        BuddyID:         instance.BuddyID,
        MediaURLs:       mediaURLs,
        Note:            req.Note,
        Status:          'pending',
        ExpiresAt:       expiresAt,
    })
    if err != nil { return nil, err }
    // 4. Notify buddy
    s.events.Publish(ctx, 'proof.submitted', ProofSubmittedEvent{
        ProofID: proof.ID, BuddyID: instance.BuddyID,
        TaskName: instance.Title, ExpiresAt: expiresAt,
    })
    return proof, nil
}
```

9.2 Verification Decision & Auto-Approve Job

```
// POST /v1/proofs/:id/review
func (s *VerificationService) ReviewProof(ctx context.Context, req
ReviewRequest) error {
    proof, _ := s.repo.GetProof(ctx, req.ProofID)
```

```
if proof.BuddyID != req.BuddyID { return errcode.ErrForbidden }
if proof.Status != 'pending'      { return errcode.ErrAlreadyReviewed }
newStatus := 'approved'
if !req.Approved { newStatus = 'rejected' }
s.repo.UpdateProofStatus(ctx, proof.ID, newStatus, req.Comment)
subject := 'proof.approved'
if !req.Approved { subject = 'proof.rejected' }
s.events.Publish(ctx, subject, ProofReviewedEvent{
    ProofID: proof.ID, TaskInstanceID: proof.TaskInstanceID,
    SubmitterID: proof.SubmitterID, BuddyID: proof.BuddyID,
    Approved: req.Approved,
})
return nil
}

// Background job: auto-approve expired proofs (runs every 15 min)
func (s *VerificationService) ProcessExpiredProofs(ctx context.Context) error {
    expired, _ := s.repo.GetExpiredPendingProofs(ctx)
    for _, proof := range expired {
        s.repo.UpdateProofStatus(ctx, proof.ID, 'auto approved', 'Buddy did not
review in time')
        // Penalise buddy verification score
        s.events.Publish(ctx, 'buddy.verification.missed',
BuddyVerificationMissedEvent{
            BuddyID: proof.BuddyID, Penalty: 5,
        })
        // Award XP as if approved
        s.events.Publish(ctx, 'proof.approved', ProofReviewedEvent{
            ProofID: proof.ID, TaskInstanceID: proof.TaskInstanceID,
            SubmitterID: proof.SubmitterID, Approved: true,
        })
    }
    return nil
}
```

10. notification-service

10.1 Notification Event Subscriptions

NATS Subject	Trigger	Notification Type	Channel(s)
task.reminder.due	Cron job 15min before task window	Task Reminder	Push, In-App
proof.submitted	Proof uploaded by user	New Proof to Review	Push, In-App
proof.expiring	Proof 2hrs from auto-approve	Verification Expiring	Push
proof.approved	Buddy approves proof	Task Verified + XP Earned	Push, In-App
proof.rejected	Buddy rejects proof	Proof Rejected	Push, In-App
buddy.assigned	New buddy assignment	Meet Your Buddy	Push, In-App
buddy.rotated	Buddy rotation completed	Buddy Changed	Push, In-App
user.levelup	Level threshold crossed	Level Up!	Push, In-App (celebration)
xp.awarded	XP/coins credited	XP Earned	In-App toast only
community.joined	User joins community	Welcome to Community	In-App
streak.broken	Missed task, streak 0	Streak Lost	Push, In-App
leaderboard.rank.top3	Month-end ranking	You're in the Top 3!	Push, In-App

10.2 Push Notification Dispatcher

```
// service/push.go
func (s *NotificationService) SendPush(ctx context.Context, userID uuid.UUID,
n PushPayload) error {
    // 1. Fetch user's FCM tokens (multiple devices supported)
    tokens, err := s.repo.GetFCMTokens(ctx, userID)
    if err != nil || len(tokens) == 0 { return nil } // no tokens, skip
    // 2. Check user notification prefs
    prefs, _ := s.repo.GetNotifPrefs(ctx, userID)
    if !prefs.IsEnabled(n.Type) { return nil }
    // 3. Build FCM message
    msg := &messaging.MulticastMessage{
        Tokens: tokens,
        Notification: &messaging.Notification{
            Title: n.Title, Body: n.Body,
        },
        Data: n.Data,
        Webpush: &messaging.WebpushConfig{
```

```
        FCMOptions: &messaging.WebpushFCMOptions{Link: n.ActionURL},
    },
}
resp, err := s.fcm.SendEachForMulticast(ctx, msg)
// 4. Clean up invalid tokens
for i, r := range resp.Responses {
    if !r.Success && messaging.IsRegistrationTokenNotRegistered(r.Error) {
        s.repo.DeleteFCMToken(ctx, tokens[i])
    }
}
return err
}
```

10.3 In-App Notification Feed

```
// Notifications stored in PostgreSQL, served via REST
// GET /v1/notifications?limit=20&cursor=<token>

-- notifications table
CREATE TABLE notifications (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID NOT NULL REFERENCES users(id),
    type        TEXT NOT NULL,
    title       TEXT NOT NULL,
    body        TEXT,
    data        JSONB,
    action_url  TEXT,
    is_read     BOOLEAN DEFAULT false,
    created_at  TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, is_read,
created_at DESC);
```

11. store-service

11.1 Purchase Transaction

```
// POST /v1/store/purchase/:itemId
func (s *StoreService) PurchaseItem(ctx context.Context, userID, itemID
uuid.UUID) error {
    // All in a single DB transaction for atomicity
    return s.repo.WithTx(ctx, func(tx pgx.Tx) error {
        // 1. Lock item row (prevent race on limited items)
        item, err := s.repo.GetItemForUpdate(ctx, tx, itemID)
        if err != nil { return err }
        if item.IsLimited && item.Stock <= 0 { return errcode.ErrOutOfStock }
        if !item.IsAvailableNow() { return errcode.ErrItemNotAvailable }
        // 2. Deduct coins atomically (returns error if insufficient)
        if err := s.repo.DeductCoins(ctx, tx, userID, item.CoinCost); err != nil {
            return errcode.ErrInsufficientCoins
        }
        // 3. Add to user inventory
        if err := s.repo.AddToInventory(ctx, tx, userID, itemID); err != nil {
            return err
        }
        // 4. Decrement stock if limited
        if item.IsLimited {
            s.repo.DecrStock(ctx, tx, itemID)
        }
        return nil
    })
}
```

12. Database Design

12.1 Full Schema — All Tables

```
-- -- Users -----
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL UNIQUE,
  display_name TEXT NOT NULL,
  avatar_url  TEXT,
  level       INT DEFAULT 1,
  total_xp    BIGINT DEFAULT 0,
  coins       BIGINT DEFAULT 0,
  streak_current INT DEFAULT 0,
  streak_best  INT DEFAULT 0,
  last_active_date DATE,
  solo_mode   BOOLEAN DEFAULT false,
  timezone_offset INT DEFAULT 0,    -- minutes from UTC
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- -- Goals -----
CREATE TABLE goals (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  community_id UUID REFERENCES communities(id) ON DELETE SET NULL,
  library_ref  UUID REFERENCES goal_library(id),
  title       TEXT NOT NULL,
  description  TEXT,
  category    TEXT,
  privacy     TEXT NOT NULL CHECK(privacy IN
('private','community','public')),
  status      TEXT NOT NULL DEFAULT 'active' CHECK(status IN
('active','paused','completed','archived')),
  target_date  DATE NOT NULL,
  start_date   DATE NOT NULL DEFAULT CURRENT_DATE,
  progress_pct DECIMAL(5,2) DEFAULT 0,
  tags        TEXT[] DEFAULT '{}',
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_goals_user_status ON goals(user_id, status);

-- -- Tasks -----
CREATE TABLE tasks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  goal_id     UUID NOT NULL REFERENCES goals(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES users(id),
  title       TEXT NOT NULL,
  description  TEXT,
  type        TEXT NOT NULL CHECK(type IN ('one_time','recurring')),
  recurrence_rule JSONB,          -- null for one_time
```

```
last_expanded_at    TIMESTAMPTZ,    -- for expander job
xp_value            INT NOT NULL DEFAULT 10,
coin_value          INT NOT NULL DEFAULT 10,
is_mandatory        BOOLEAN DEFAULT false,
proof_required      BOOLEAN DEFAULT false,
proof_type          TEXT CHECK(proof_type IN ('photo','text','link','any')),
verification_window_hrs INT DEFAULT 24,
tags                TEXT[] DEFAULT '{}',
priority            TEXT DEFAULT 'medium' CHECK(priority IN
('low','medium','high')),
created_at          TIMESTAMPTZ DEFAULT now(),
updated_at          TIMESTAMPTZ DEFAULT now()
);

-- — Task Instances (materialised recurring occurrences) —
CREATE TABLE task_instances (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id           UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  goal_id           UUID NOT NULL REFERENCES goals(id),
  user_id           UUID NOT NULL REFERENCES users(id),
  buddy_id          UUID REFERENCES users(id),
  scheduled_at      TIMESTAMPTZ NOT NULL,
  duration_mins     INT,
  status            TEXT NOT NULL DEFAULT 'scheduled'
  CHECK(status IN
('scheduled','pending','completed_unverified',
'completed_verified','skipped','missed','failed')),
  completed_at      TIMESTAMPTZ,
  xp_value          INT NOT NULL,
  coin_value        INT NOT NULL,
  proof_required    BOOLEAN,
  created_at        TIMESTAMPTZ DEFAULT now()
) PARTITION BY RANGE (scheduled_at); -- monthly partitions
CREATE INDEX idx_ti_user_status_date ON task_instances(user id, status,
scheduled_at);
CREATE INDEX idx_ti_buddy            ON task_instances(buddy_id, status);

-- — Communities —
CREATE TABLE communities (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name              TEXT NOT NULL,
  description        TEXT,
  avatar_url        TEXT,
  privacy            TEXT NOT NULL DEFAULT 'open'
  CHECK(privacy IN ('open','invite_only','private')),
  max_members       INT DEFAULT 500,
  buddy_rotation_days INT DEFAULT 14,
  buddy_xp_ratio     DECIMAL(4,2) DEFAULT 0.30,
  coin_solo_ratio    DECIMAL(4,2) DEFAULT 0.60,
  created_by         UUID NOT NULL REFERENCES users(id),
  created_at         TIMESTAMPTZ DEFAULT now(),
  updated_at         TIMESTAMPTZ DEFAULT now()
);
```



```
-- — Community Members —————
CREATE TABLE community_members (
  community_id  UUID NOT NULL REFERENCES communities(id),
  user_id       UUID NOT NULL REFERENCES users(id),
  role         TEXT NOT NULL DEFAULT 'member' CHECK(role IN
('admin','moderator','member')),
  joined_at    TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (community_id, user_id)
);

-- — Buddy Assignments —————
CREATE TABLE buddy_assignments (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  community_id  UUID NOT NULL REFERENCES communities(id),
  goal_id       UUID NOT NULL REFERENCES goals(id),
  user_id       UUID NOT NULL REFERENCES users(id),
  buddy_id      UUID NOT NULL REFERENCES users(id),
  rotation_start TIMESTAMPTZ NOT NULL,
  rotation_end  TIMESTAMPTZ NOT NULL,
  is_active     BOOLEAN DEFAULT true,
  verification_score DECIMAL(5,2) DEFAULT 100.0,
  created_at    TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_ba_active ON buddy_assignments(user_id, is_active,
rotation_end);

-- — Proof Submissions —————
CREATE TABLE proof_submissions (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_instance_id UUID NOT NULL REFERENCES task_instances(id),
  submitter_id  UUID NOT NULL REFERENCES users(id),
  buddy_id      UUID NOT NULL REFERENCES users(id),
  media_urls    TEXT[],
  note          TEXT,
  status        TEXT NOT NULL DEFAULT 'pending'
CHECK(status IN
('pending','approved','rejected','auto_approved','appealed')),
  buddy_comment TEXT,
  submitted_at  TIMESTAMPTZ DEFAULT now(),
  reviewed_at   TIMESTAMPTZ,
  expires_at    TIMESTAMPTZ NOT NULL
);
CREATE INDEX idx_proof_buddy_pending ON proof_submissions(buddy_id, status,
expires_at);

-- — XP Ledger (append-only, partitioned) —————
CREATE TABLE xp_ledger (
  id            UUID DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL,
  amount        INT NOT NULL,
  type          TEXT NOT NULL,
  ref_id        UUID,
  community_id  UUID,
  created_at    TIMESTAMPTZ DEFAULT now()
) PARTITION BY RANGE (created_at);
```

```
-- Create monthly partitions via migration script

-- — Store Items —————
CREATE TABLE store_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  description  TEXT,
  category    TEXT NOT NULL CHECK(category IN
('title','avatar','background','card_theme','goal_badge')),
  rarity      TEXT NOT NULL CHECK(rarity IN
('common','rare','epic','legendary')),
  coin_cost   INT NOT NULL,
  asset_url   TEXT,
  is_limited  BOOLEAN DEFAULT false,
  stock       INT,
  available_from TIMESTAMPTZ,
  available_until TIMESTAMPTZ
);

-- — User Inventory —————
CREATE TABLE user_inventory (
  user_id     UUID NOT NULL REFERENCES users(id),
  item_id     UUID NOT NULL REFERENCES store_items(id),
  is_equipped BOOLEAN DEFAULT false,
  acquired_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, item_id)
);

-- — Goal Library —————
CREATE TABLE goal_library (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  description  TEXT,
  category    TEXT,
  difficulty   TEXT CHECK(difficulty IN
('beginner','intermediate','advanced')),
  estimated_days INT,
  adoption_count BIGINT DEFAULT 0,
  tags        TEXT[],
  tasks_template JSONB,  -- Array of task configs to fork
  created_by   UUID REFERENCES users(id),
  is_curated   BOOLEAN DEFAULT false,
  created_at   TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_library_tags ON goal_library USING GIN(tags);

-- — Tags —————
CREATE TABLE tags (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  color       TEXT,
  scope       TEXT NOT NULL CHECK(scope IN ('user','community')),
  owner_id    UUID NOT NULL,  -- user_id or community_id
  created_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE (scope, owner_id, name)
```

```
);

-- -- FCM Tokens -----
CREATE TABLE fcm_tokens (
  user_id      UUID NOT NULL REFERENCES users(id),
  token        TEXT NOT NULL,
  device_type   TEXT,
  created_at    TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, token)
);
```

12.2 Row-Level Security Policies

```
-- Enable RLS on all user-data tables
ALTER TABLE goals ENABLE ROW LEVEL SECURITY;
ALTER TABLE task_instances ENABLE ROW LEVEL SECURITY;
ALTER TABLE proof_submissions ENABLE ROW LEVEL SECURITY;

-- goals: user sees own goals + community goals they're a member of
CREATE POLICY goals_access ON goals
  FOR ALL USING (
    user_id = current_setting('app.user_id')::UUID
  OR (
    community_id IS NOT NULL
    AND privacy != 'private'
    AND EXISTS (
      SELECT 1 FROM community_members cm
      WHERE cm.community_id = goals.community_id
            AND cm.user_id = current_setting('app.user_id')::UUID
    )
  )
);

-- Set app.user_id in every query (done by pkg/db middleware)
-- pool.Exec(ctx, 'SET LOCAL app.user_id = $1', userID)
```

13. Event System Design (NATS JetStream)

13.1 Stream Configuration

```
// Stream: GOALKEEPER
// All subjects prefixed: goalkeeper.*.*
StreamConfig{
  Name:      'GOALKEEPER',
  Subjects:  ['goalkeeper.>'],
  Retention: nats.WorkQueuePolicy,
  MaxAge:    7 * 24 * time.Hour, // Events retained 7 days for replay
  MaxBytes:  5 * 1024 * 1024 * 1024, // 5 GB max
  Replicas:  3, // Production: 3-node cluster
}
```

13.2 Full Event Catalogue

Subject	Publisher	Consumers	Payload Key Fields
goalkeeper.task.completed	tasks-svc	gamification-svc, analytics-svc	userID, taskInstanceID, xpValue, coinValue, isSolo, communityID, goalID
goalkeeper.task.missed	tasks-svc	gamification-svc (streak), notification-svc	userID, taskInstanceID, goalID, buddyID
goalkeeper.proof.submitted	verification-svc	notification-svc	proofID, taskInstanceID, buddyID, submitterID, expiresAt
goalkeeper.proof.approved	verification-svc	tasks-svc, gamification-svc, notification-svc	proofID, taskInstanceID, submitterID, buddyID
goalkeeper.proof.rejected	verification-svc	tasks-svc, notification-svc	proofID, taskInstanceID, submitterID, buddyComment
goalkeeper.proof.auto_approved	verification-svc	tasks-svc, gamification-svc, notification-svc	proofID, buddyID, penaltyPoints
goalkeeper.xp.awarded	gamification-svc	community-svc (leaderboard), notification-svc, analytics-svc	userID, amount, coins, multiplier, communityID, type
goalkeeper.user.levelup	gamification-svc	notification-svc, users-svc	userID, oldLevel, newLevel, bonusCoins
goalkeeper.buddy.assigned	community-svc	notification-svc	userID, buddyID, communityID, goalID, rotationEnd

goalkeeper.buddy.rotated	community-svc	notification-svc	userID, buddyID, oldBuddyID, communityID
goalkeeper.goal.progress.recalc	tasks-svc	goals-svc	goalID, userID
goalkeeper.goal.completed	goals-svc	gamification-svc, notification-svc, analytics-svc	goalID, userID, communityID
goalkeeper.streak.broken	gamification-svc	notification-svc, analytics-svc	userID, previousStreak
goalkeeper.library.adopted	goals-svc	search-svc (incr adoption count)	libGoalID, userID

14. Caching Strategy

Cache Key Pattern	Data Cached	TTL	Invalidation
user:{userID}:profile	User profile snapshot	5 min	On PATCH /users/me
user:{userID}:gamification	XP, coins, level, streak	30 sec	On xp.awarded event
lb:{communityID}:{goalID}:{window}	Leaderboard ZSET (sorted set)	Rolling (Expire 90d)	ZINCRBY on xp.awarded event
community:{communityID}:meta	Community config + member count	10 min	On community update
store:items:{category}	Store item catalogue	1 hour	On store admin update
goal:library:{page}:{tags}	Goal library search results	15 min	On library.adopted count update
session:{jti}:revoked	Revoked JWT JTI set	Until token expiry	On logout / account deactivation
proof:{proofID}:pending	Proof pending review	Until reviewed or expired	On proof review

15. Background Jobs

Job Name	Service	Schedule	Description
task-reminder-dispatcher	notification-svc	Every 5 min	Query task_instances with scheduled_at in next 15 min, status=scheduled. Fire reminders. Mark reminder_sent=true.
recurrence-expander	tasks-svc	Every 24h at 01:00 UTC	Expand recurring tasks into task_instances for next 90 days.
task-window-opener	tasks-svc	Every 2 min	Move task_instances from 'scheduled' to 'pending' when scheduled_at <= now.
task-missed-checker	tasks-svc	Every 5 min	Move 'pending' instances to 'missed' if window elapsed. Publish task.missed event.
proof-auto-approver	verification-svc	Every 15 min	Auto-approve proof_submissions past expires_at. Penalise buddy.
buddy-rotation-processor	community-svc	Every 1h	Find buddy_assignments with rotation_end <= now. Trigger rotation.
streak-daily-reset	gamification-svc	Daily 00:05 UTC	Zero streak for users with no completed task yesterday.
leaderboard-monthly-close	community-svc	1st of month 00:10 UTC	Award top-3 coins. Archive ZSET for past month. Publish rank events.
analytics-aggregator	analytics-svc	Every 1h	Aggregate raw events into time-bucketed metrics (hourly/daily rollups).
search-indexer	search-svc	Every 30 min	Sync new/updated goals to Typesense. Process library additions.
proof-media-gc	verification-svc	Weekly Sunday 03:00 UTC	Delete media from object storage for proofs older than retention period.

16. Security Implementation

16.1 JWT Authentication Flow

```
// Every request passes through auth middleware
func AuthMiddleware(jwtVerifier *auth.Verifier) echo.MiddlewareFunc {
    return func(next echo.HandlerFunc) echo.HandlerFunc {
        return func(c echo.Context) error {
            token := extractBearerToken(c.Request())
            if token == '' { return c.JSON(401, errcode.ErrUnauthorized) }
            claims, err := jwtVerifier.Verify(token) // RS256, checks exp, iss,
aud
            if err != nil { return c.JSON(401, errcode.ErrTokenExpired) }
            // Check revocation (Redis set: session:{jti}:revoked)
            if jwtVerifier.IsRevoked(c.Request().Context(), claims.JTI) {
                return c.JSON(401, errcode.ErrTokenRevoked)
            }
            // Inject userID into request context + set Postgres RLS variable
            ctx := context.WithValue(c.Request().Context(), middleware.UserIDKey,
claims.Sub)
            c.SetRequest(c.Request().WithContext(ctx))
            return next(c)
        }
    }
}
```

16.2 Input Validation

```
// All request bodies validated with go-validator before reaching service
layer
type CreateGoalRequest struct {
    Title      string `json:'title'`    validate:'required,min=1,max=120'`
    Description string `json:'description'` validate:'omitempty,max=1000'`
    TargetDate string `json:'targetDate'` validate:'required,datetime=2006-01-
02,future'`
    Privacy    string `json:'privacy'`  validate:'required,oneof=private
community public'`
    Tags       []string `json:'tags'`    validate:'dive,max=30'`
}

func (h *GoalHandler) Create(c echo.Context) error {
    var req CreateGoalRequest
    if err := c.Bind(&req); err != nil { return c.JSON(400,
errcode.ErrBadRequest) }
    if err := validator.Struct(req); err != nil {
        return c.JSON(422, formatValidationError(err))
    }
    // ... proceed
}
```


16.3 Rate Limiting

```
// Redis token bucket - 100 req/min per authenticated user
func RateLimitMiddleware(redis *redis.Client) echo.MiddlewareFunc {
    return func(next echo.HandlerFunc) echo.HandlerFunc {
        return func(c echo.Context) error {
            userID := middleware.UserIDFromCtx(c.Request().Context())
            key := fmt.Sprintf('rl:user:%s', userID)
            count, err := redis.Incr(c.Request().Context(), key).Result()
            if count == 1 {
                redis.Expire(c.Request().Context(), key, time.Minute)
            }
            if count > 100 {
                c.Response().Header().Set('Retry-After', '60')
                return c.JSON(429, errcode.ErrRateLimited)
            }
            return next(c)
        }
    }
}
```

17. Observability & Monitoring

17.1 Logging Convention

```
// Every log line is structured JSON (zerolog)
// Request log example:
{
  "level": "info",
  "service": "tasks-svc",
  "trace_id": "abc123",
  "span_id": "def456",
  "user_id": "uuid-...",
  "method": "POST",
  "path": "/v1/tasks/uuid/complete",
  "status": 200,
  "latency_ms": 42,
  "timestamp": "2025-02-01T12:00:00Z"
}
```

17.2 Key Metrics (Prometheus)

Metric Name	Type	Labels	Purpose
http_request_duration_seconds	Histogram	service, method, path, status	API latency tracking; SLA compliance
task_completion_total	Counter	service, is_solo, proof_required	Task completion rate by mode
xp_awarded_total	Counter	service, type	XP economy health tracking
proof_auto_approve_total	Counter	service	Buddy engagement monitoring
nats_event_lag_seconds	Gauge	stream, consumer	Event processing backlog alerting
db_query_duration_seconds	Histogram	service, query_name	Slow query detection
redis_cache_hit_ratio	Gauge	service, cache_key_prefix	Cache efficiency monitoring
active_buddy_assignments_total	Gauge	community_id	Community health signal

17.3 Alerting Rules

Alert Name	Condition	Severity	Action
HighAPILatency	p95 latency > 1s for 5 min	Warning	PagerDuty notify on-call
CriticalAPILatency	p95 latency > 3s for 2 min	Critical	PagerDuty page on-call immediately

ProofAutoApproveSpike	auto_approve rate > 30% in 1h	Warning	Investigate buddy engagement
NATSConsumerLag	event lag > 1000 messages	Warning	Scale consumer pods
DBConnectionPool	pool utilisation > 80%	Warning	Scale DB connections or read replica
LowDiskOnDB	disk > 80% full	Critical	Expand PVC immediately
TaskMissedSpike	missed rate > 50% in 1h	Warning	Check task-window-opener job health

18. Deployment Architecture

18.1 Kubernetes Pod Specifications

Service	CPU Request	CPU Limit	Memory Req	Memory Limit	Min Replicas	Max Replicas	HPA Metric
users-svc	100m	500m	128Mi	512Mi	2	10	CPU > 60%
goals-svc	100m	500m	128Mi	512Mi	2	10	CPU > 60%
tasks-svc	200m	1000m	256Mi	1Gi	3	20	CPU > 70%
community-svc	150m	750m	256Mi	1Gi	2	15	CPU > 60%
gamification-svc	100m	500m	128Mi	512Mi	2	10	NATS lag > 500
verification-svc	150m	750m	256Mi	1Gi	2	15	CPU > 65%
notification-svc	100m	500m	128Mi	512Mi	2	15	NATS lag > 200
store-svc	50m	250m	64Mi	256Mi	1	5	CPU > 60%
search-svc	100m	500m	128Mi	512Mi	1	5	CPU > 60%
analytics-svc	200m	1000m	512Mi	2Gi	1	5	CPU > 70%

18.2 CI/CD Pipeline

```
# .github/workflows/deploy.yml (simplified)
on: [push to main]
jobs:
  test:
    - go test ./...           # Unit + integration tests
    - npx vitest run          # Frontend tests
    - gosec ./...             # Go security scan
    - eslint + tsc --noEmit    # Frontend lint + type check

  build:
    - docker build -t service:sha .
    - docker push to registry

  migrate:
    - flyway migrate -url=$DB_URL # Run pending SQL migrations
```

```
deploy:
  - kubectl set image deploy/tasks-svc tasks=$IMAGE:$SHA
  - kubectl rollout status deploy/tasks-svc --timeout=5m
  - on failure: kubectl rollout undo deploy/tasks-svc
```

19. Inter-Service Communication

19.1 Synchronous gRPC (Internal)

Services that need real-time data from another service use gRPC over an internal Kubernetes ClusterIP service. This keeps sensitive internal APIs off the public internet.

```
// proto/users.proto
service UsersInternal {
  rpc GetUserProfile (GetUserRequest) returns (UserProfile);
  rpc GetUsersBatch (GetUsersBatchRequest) returns (UsersBatchResponse);
}

// proto/gamification.proto
service GamificationInternal {
  rpc GetUserSnapshot (GetSnapshotRequest) returns (GamificationSnapshot);
}

// Callers:
// community-svc calls users-svc via gRPC to hydrate leaderboard entries
// notification-svc calls users-svc via gRPC to get FCM tokens + prefs
// gamification-svc calls users-svc via gRPC to get level before awarding XP
```

19.2 Communication Matrix

From Service	To Service	Protocol	When
community-svc	users-svc	gRPC	Hydrate leaderboard with user profiles
notification-svc	users-svc	gRPC	Get FCM tokens + notification prefs
gamification-svc	users-svc	gRPC	Read level snapshot before XP award
goals-svc	tasks-svc	gRPC	Get task instance counts for progress calc
verification-svc	tasks-svc	gRPC	Get task instance details on proof submit
verification-svc	community-svc	gRPC	Get buddy ID for a task instance
search-svc	goals-svc	gRPC	Pull new/updated goals for Typesense sync
analytics-svc	(all)	NATS consume	Ingests all domain events for aggregation
tasks-svc	gamification-svc	NATS publish	task.completed, task.missed events
verification-svc	gamification-svc	NATS publish	proof.approved events for XP award

20. Testing Strategy

20.1 Frontend Testing

Test Type	Tool	Scope	Coverage Target
Unit Tests	Vitest + Testing Library	Hooks, utility functions, store actions	90% line coverage
Component Tests	Vitest + Testing Library + MSW	Individual components with mocked API	Key user interactions
Integration Tests	Playwright	Full user flows: create goal → task → complete	Critical paths P0
Visual Regression	Storybook + Chromatic	Design system components	All GK* components
PWA Audit	Lighthouse CI	Performance, PWA score, accessibility	Score > 85 all categories

20.2 Backend Testing

Test Type	Tool	Scope	Coverage Target
Unit Tests	Go testing + testify	Service layer business logic, pure functions	85%+ coverage
Repository Tests	Go testing + testcontainers	Postgres queries against real DB in Docker	All repo methods
Handler Tests	Go testing + httptest	Request parsing, validation, response format	All handlers
Integration Tests	Go testing + testcontainers	Full service flow incl. NATS events	P0 flows
Load Tests	k6	API endpoints under sustained traffic	p95 < 300ms at 1000 RPS
Contract Tests	Pact	API consumer-producer contracts	All inter-service APIs

21. OpenAPI 3.1 Spec Excerpt

```
openapi: 3.1.0
info:
  title: GoalKeeper API
  version: 1.0.0
  description: GoalKeeper backend REST API

paths:
  /v1/tasks/{taskId}/complete:
    post:
      summary: Mark a task complete and optionally upload proof
      tags: [Tasks]
      security: [{bearerAuth: []}]
      parameters:
        - name: taskId
          in: path
          required: true
          schema: { type: string, format: uuid }
      requestBody:
        content:
          multipart/form-data:
            schema:
              type: object
              properties:
                note: { type: string, maxLength: 500 }
                files:
                  type: array
                  items: { type: string, format: binary }
                  maxItems: 3
      responses:
        '200': { description: Task marked complete }
        '400': { $ref: '#/components/responses/BadRequest' }
        '409': { description: Task not in pending state }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /v1/communities/{communityId}/leaderboard:
    get:
      summary: Get community leaderboard
      tags: [Community]
      security: [{bearerAuth: []}]
      parameters:
        - name: communityId
          in: path
          required: true
          schema: { type: string, format: uuid }
        - name: window
          in: query
          schema: { type: string, enum: [all, month, week], default: month }
        - name: limit
          in: query
          schema: { type: integer, minimum: 1, maximum: 100, default: 50 }
      responses:
        '200':
```



```
    description: Leaderboard entries
    content:
      application/json:
        schema:
          type: array
          items: { $ref: '#/components/schemas/LeaderboardEntry' }

components:
  schemas:
    LeaderboardEntry:
      type: object
      properties:
        rank: { type: integer }
        userId: { type: string, format: uuid }
        displayName: { type: string }
        avatarUrl: { type: string }
        level: { type: integer }
        communityXP: { type: integer }
        goalCompletion: { type: number, format: float }
        streak: { type: integer }
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

22. PRD → LLD Traceability Matrix

PRD Requirement	LLD Section	Implementation
FR-GM-01: Goal Creation	§5 goals-service	GoalService.CreateGoal + handler + schema
FR-GM-03: Goal Library	§5.2 AdoptLibraryGoal	Fork pattern, goal_library table, search-svc indexing
FR-TM-01: Task Configuration	§6 tasks-service	tasks + task_instances tables, RecurrenceRule struct
FR-TM-02: Task States	§6.2 State Machine	state_machine.go, NATS events for each transition
FR-TM-03: Task Tags	§12.1 tags table	GIN-indexed TEXT[] on tasks, tags table for management
FR-CAL-01: Calendar Drag-Drop	§2.4.2, §2.8	DndContext + BigCalendar, handleReschedule, RescheduleModal
FR-GAM-01: XP Economy	§8.1 XP Engine	handleTaskCompleted, streakMultiplier, solo penalty
FR-GAM-02: Level System	§8.2 Level Computation	XPToLevel formula, checkAndApplyLevelUp
FR-GAM-03: Reward Store	§11 store-service	PurchaseItem tx, user_inventory, store_items table
FR-COM-01: Community Creation	§7 community-service	CommunityService.Create, community_members table
FR-COM-02: Buddy System	§7.1 Buddy Assigner	AssignBuddy, scoreCandidates, buddy_assignments table
FR-COM-03: Proof Verification	§9 verification-service	SubmitProof, ReviewProof, proof_submissions table
FR-COM-04: Leaderboard	§7.3 Leaderboard	Redis ZSET, HandleXPAwarded, GetLeaderboard
NFR-P01: API < 300ms	§14 Caching, §18.1	Redis user/gamification cache, read replica routing
NFR-P02: Leaderboard < 100ms	§7.3, §14	Redis ZSET ZRevRangeWithScores, pre-computed scores
NFR-S01: 100k CCU	§18.1 Kubernetes HPA	Stateless pods, HPA on CPU/NATS lag metrics
NFR-SEC02: RLS	§12.2 RLS Policies	Postgres RLS, SET LOCAL app.user_id per query
NFR-R05: PWA Offline	§2.7 Service Worker	Workbox, IndexedDB queue, Background Sync API
FR-4.6: Notifications	§10 notification-svc	NATS consumer, FCM push, in-app notifications table
11.1 AI Architecture	§1.1, §13.2 analytics-svc	analytics-svc consumes all events, feeds future AI gRPC

Document End — GoalKeeper LLD v1.0 | *All rights reserved. Internal and Confidential.*